

Application de Gestion Bancaire (Java / JavaFX)

Matisse
kra
bts sio
Sommaire

1. Introduction et Contexte	2
2. Objectifs Pédagogiques et Techniques	2
3. Architecture de l'Application (Modèle MVC)	2
A. Le Modèle (Package Model)	2
B. La Vue (Package View / Dossier resources)	2
C. Le Contrôleur (Package controller)	4
4. Difficultés Rencontrées et Solutions Apportées	6
5. Conclusion	6
1. Introduction et Contexte	2
2. Objectifs Pédagogiques et Techniques	2
3. Architecture de l'Application (Modèle MVC)	2
A. Le Modèle (Package Model)	2
B. La Vue (Package View / Dossier resources)	2
C. Le Contrôleur (Package controller)	4
4. Difficultés Rencontrées et Solutions Apportées	6
5. Conclusion	6

1. Introduction et Contexte

L'objectif de ce projet pratique était de concevoir et développer une application de gestion bancaire avec une interface graphique de bureau. Ce projet a permis de mettre en application les concepts fondamentaux de la Programmation Orientée Objet (POO) en Java, ainsi que la création d'interfaces graphiques (GUI) à l'aide de JavaFX et de l'outil Scene Builder, le tout structuré selon l'architecture logicielle MVC (Modèle-Vue-Contrôleur).

2. Objectifs Pédagogiques et Techniques

- Maîtriser les concepts de la POO : Encapsulation, Héritage, Classes Abstraites et Polymorphisme.
- Concevoir une architecture MVC stricte séparant la logique métier, l'affichage et les interactions.
- Manipuler les composants graphiques JavaFX (FXML, Stage, Scene, FXMLLoader).
- Gérer les événements utilisateurs (clics, sélection de listes, saisie de formulaires).
- Appréhender l'outil de gestion de projet Maven (compilation, gestion des dépendances, cycle de vie).

3. Architecture de l'Application (Modèle MVC)

Le projet a été découpé de manière rigoureuse en trois paquetages distincts :

A. Le Modèle (Package Model)

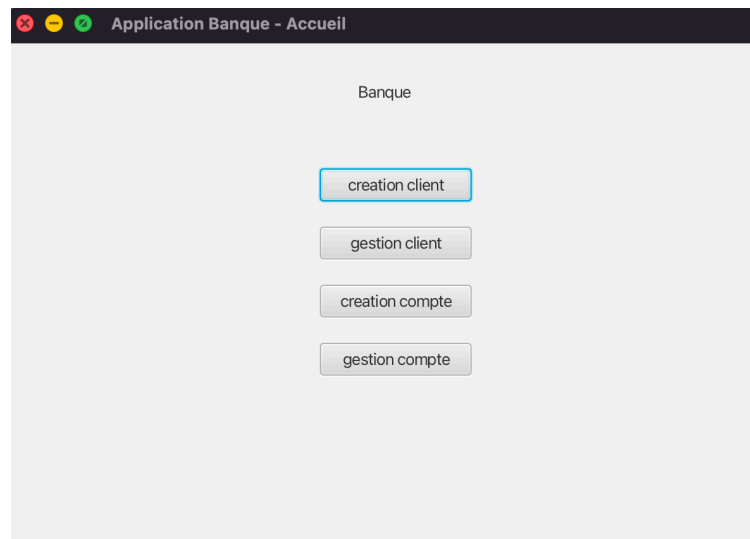
Cette partie contient la logique métier et la définition des entités que l'on a vue dans le tp héritage qui se trouve dans le portfolio mais voici un rappel des classe :

- Classe Client : Permet d'instancier des clients avec leurs informations personnelles (identifiant, nom, prénom, genre, date de naissance, coordonnées).
- Classe Abstraite Compte : Définit les attributs communs à tous les comptes (numéro, titulaire, solde, devise). Elle impose l'encapsulation via des modificateurs d'accès privés (private) et des accesseurs/mutateurs (getters/setters). Elle déclare également une méthode abstraite debiter().
- Héritage et Polymorphisme (CompteCourant & Epargne) : Ces deux classes héritent de la classe mère Compte. L'application du polymorphisme se traduit par la redéfinition (@Override) de la méthode debiter() pour appliquer des règles de gestion spécifiques (gestion du découvert pour le compte courant, et limite de retrait à la moitié du solde pour l'épargne).
- Classe App : Classe principale servant de point d'entrée et stockant les données globales de l'application dans des listes statiques (listeDesClients, tousLesComptes).

B. La Vue (Package View / Dossier resources)

Les interfaces ont été conçues de manière déclarative en XML via des fichiers .fxml manipulés avec Scene Builder. L'application se compose d'un menu d'accueil permettant d'invoquer quatre fenêtres secondaires :

1. la page d'accueil



2. Création de client (formulaires, boutons radio, sélecteur de date).

3. Gestion des clients (modification des coordonnées).

4. Création de compte (liaison avec un client via une liste déroulante).

CRÉATION DE COMPTE

Sélectionner Client: Choisir un client...

Type de compte : ☐ Compte Courant ☐ Compte Epargne

n° CB:

Découvert Max: Taux:

SOLDE:

CRÉER

5. Gestion des comptes (opérations de crédit, débit et modification des plafonds/taux).

GESTION DES COMPTES

1. Sélection

Client: Choisir un client...

Compte: Choisir un compte...

2. Informations et Modification

N° Compte: Solde (€):

Découvert/Taux: Enregistrer les modifications

3. Opérations (Crédit / Débit)

Montant: Ex: 50.0 Créditer Débit

C. Le Contrôleur (Package controller)

Chaque vue FXML est pilotée par son propre contrôleur Java. Ces contrôleurs récupèrent les données saisies,instancient les objets du modèle, et mettent à jour les vues. Des blocs try...catch ont été implémentés pour assurer la robustesse de l'application face aux erreurs de saisie (ex: NumberFormatException lors de la saisie d'un texte à la place d'un nombre pour un solde). Pour fluidifier l'expérience utilisateur, chaque contrôleur secondaire intègre une logique de fermeture automatique de sa fenêtre (Stage.close()) une fois l'opération validée.

Exemple avec la page d'accueil :

Dans cette page d'accueil, j'ai créé différentes méthodes dans la classe AccueilController.java afin de piloter les interactions de l'utilisateur. Chaque bouton de l'interface est lié à une méthode spécifique (annotée @FXML) qui utilise le service FXMLLoader pour charger dynamiquement la vue correspondante (ex: CreationClient.fxml ou GestionCompte.fxml). Ce mécanisme de navigation instancie un nouveau Stage (fenêtre) et une nouvelle Scene, permettant ainsi de passer d'un module à l'autre tout en conservant le menu principal en arrière-plan. pour mes liaison fxml entre les différente Scene j'ai utilisé des fx:id pour injecter tes boutons dans le contrôleur (ex: btnCreationClient) et chaque méthode est isolée dans un bloc try...catch pour gérer les éventuelles erreurs de chargement de fichiers et éviter un plantage de l'application.

Matisse kra
bts sio

extrait du code :

```
1 package controller;
2
3 import javafx.fxml.FXML;
4 import javafx.fxml.FXMLLoader;
5 import javafx.scene.Parent;
6 import javafx.scene.Scene;
7 import javafx.scene.control.Button;
8 import javafx.stage.Stage;
9
10 public class AccueilController {
11
12     @FXML
13     private Button btnCreationClient; Variable btnCreationClient is never read
14     @FXML
15     private Button btnGestionClient; Variable btnGestionClient is never read
16     @FXML
17     private Button btnCreationCompte; Variable btnCreationCompte is never read
18     @FXML
19     private Button btnGestionCompte; Variable btnGestionCompte is never read
20
21     @FXML
22     private void handleCreationClient() { handleCreationClient is never used
23         try {
24             FXMLLoader loader = new FXMLLoader(getClass().getResource("/View/CreationClient.fxml"));
25             Parent root = loader.load();
26             Stage creationStage = new Stage();
27             creationStage.setTitle(value: "Création de Client");
28             creationStage.setScene(new Scene(root));
29             creationStage.show();
30         } catch (Exception e) { Can be replaced with multicatch or several catch clauses catching specific exceptions
31             System.out.println(x: "Impossible d'ouvrir la fenêtre de création de client."); Replace this with a more descriptive message
32             e.printStackTrace(); Print Stack Trace
33         }
34     }
35 }
```

exemple CreationClientController.java

Dans cette page de création de client, j'ai créé différentes méthodes dans la classe CreationClientController.java pour assurer la transition entre la saisie utilisateur et le modèle métier. La méthode principale, handleCreerClient(), extrait les informations des différents composants (tels que le DatePicker pour la date de naissance ou le ToggleGroup pour le genre) et les convertit dans les types de données attendus par le constructeur de la classe Client. J'ai intégré une gestion d'erreurs via un bloc try...catch pour intercepter les saisies incorrectes, comme un identifiant non numérique. Enfin, une fois l'objet instancié et ajouté à la liste globale de l'application, le contrôleur commande la fermeture automatique de la fenêtre via le Stage pour fluidifier le retour à l'accueil. »

extrait du code :

```
@FXML
private void handleCreerClient() { handleCreerClient is never used
    try {
        // 1. Récupération des données textuelles et numériques
        int id = Integer.parseInt(txtIdClient.getText());
        String nom = txtNom.getText();
        String prenom = txtPrenom.getText();

        // 2. Déduction du booléen pour le genre : on met "true" pour Homme, "false" pour Femme
        Boolean genre = radioHomme.isSelected();

        // 3. Récupération de la date avec le DatePicker
        LocalDate dateNaissance = dpDateNaissance.getValue();

        String categorie = txtCategorie.getText();
        String adresse = txtAdresse.getText();
        String tel = txtTel.getText();
        String email = txtEmail.getText();

        // 4. Instanciation du nouveau client avec ton constructeur
        Client nouveauClient = new Client(id, nom, prenom, genre, dateNaissance, categorie, adresse, tel, email);

        // 5. Ajout à la liste globale de l'application
        App.listeDesClients.add(nouveauClient);
        System.out.println("Succès : Client " + nom + " " + prenom + " créé et ajouté à la base !"); Replace th

        // 6. Nettoyage de l'interface pour la prochaine saisie
        viderChamps();

        // 7. Fermer la fenêtre automatiquement pour revenir à l'accueil
        Stage stage = (Stage) btnCreer.getScene().getWindow();
        stage.close();
    } catch (NumberFormatException e) {
        System.out.println(x: "Erreur de saisie : L'ID du client doit être un nombre entier valide."); Replace
    } catch (Exception e) {
        System.out.println(x: "Veuillez remplir correctement tous les champs obligatoires (notamment la date).");
    }
}
```

4. Difficultés Rencontrées et Solutions Apportées

Au cours du développement, plusieurs défis techniques liés à l'environnement de développement ont été surmontés :

- Problème de modules JavaFX : Des erreurs de liaison entre les fichiers FXML et les contrôleurs sont apparues au lancement. Solution : mon dossier controller ne respectait pas l'architecture imposée par maven j'ai déplacé mon dossier controller dans src/main/java
- Erreurs de compilation Maven (ClassNotFoundException) : L'application ne trouvait pas les contrôleurs après modification des noms de packages. L'environnement Maven est sensible à la casse (majuscules/minuscules) et conserve des fichiers en cache. Solution : Respect strict de la convention de nommage des packages en minuscules (ex: controller) et utilisation systématique de la commande mvn clean javafx:run (ou suppression manuelle du répertoire cible target) pour forcer une recompilation propre.
- Erreur XML ("Contenu non autorisé dans le prologue") : Plantage du chargement de l'accueil. Solution : Nettoyage de la toute première ligne du fichier .fxml pour s'assurer qu'aucun espace ou caractère invisible ne précède la balise de déclaration XML.

5. Conclusion

Ce TP a permis de consolider la conception d'une application Java complète. Le passage d'une simple console en pseudo-code à une véritable interface graphique métier démontre l'intérêt du modèle MVC. L'application est aujourd'hui fonctionnelle : elle permet d'enregistrer des clients de manière persistante (en mémoire vive) durant l'exécution, de leur attribuer

Matisse kra
bts sio

différents types de comptes, et de simuler des opérations bancaires sécurisées par les règles du modèle (polymorphisme des débits).